Autonomous Navigation with EDT Path Planning and MPPI Control

Karthik Shivashankaran
New York University Tandon School of Engineering
Brooklyn, NY, USA

***Abstract*—This report presents the design, implementation, and evaluation of a complete autonomous navigation pipeline for a four-wheeled mobile robot. The system represents the environment as an Euclidean Distance Transform (EDT) occupancy grid, uses Dijkstra's algorithm to compute a wall-clearance-aware global reference path, and applies Model Predictive Path Integral (MPPI) control to track that path while avoiding obstacles. A Monte Carlo Particle Filter (PF) with N=1,000 particles provides real-time pose estimates from RPLidar scans. The vectorized MPPI implementation samples K=100 trajectories over a horizon of T=60 steps and completes each iteration in 1.5–2.5 ms, well within the 100 ms control cycle. The PF converges from an unknown starting position within approximately 1 s of motion, and the full pipeline loop runs in 40–42 ms.**

I. Introduction

Autonomous mobile robot navigation requires a representation of the environment, a mechanism for pose estimation, global path planning, and local collision-free control. This work integrates all four into a single pipeline running at 10 Hz on a laptop communicating wirelessly with an Ackermann-steered robot.

The environment is modelled as a pre-surveyed wall polygon rasterized into an EDT grid that stores per-cell clearance to the nearest obstacle. Dijkstra's algorithm runs on a clearance-weighted graph derived from that grid to produce a globally optimal reference path whenever the operator specifies a goal. Pose estimation is performed by a Monte Carlo Particle Filter (PF) whose weighted mean drives the MPPI planner. MPPI selects steering commands by sampling trajectory perturbations and weighting them through a composite cost that combines wall proximity (via the EDT), path deviation, heading alignment, and goal distance.

Section II details the mathematical formulation of each module. Section III describes the hardware and experimental procedure. Section IV presents quantitative timing and localization results. Section V summarizes findings and future directions.

II. Method

A. System Overview

The robot state is the planar pose $\mathbf{x}_t = [x_t,\, y_t,\, \theta_t]^\top$. At each 100 ms tick the control loop executes four sequential steps:

1) Receive the latest encoder count and steering angle via UDP.
2) Update the PF with odometry and current LiDAR scan.
3) Run one MPPI iteration using the PF mean as the current state estimate.
4) Transmit the selected speed and steering command to the Arduino.

Figure 1 shows the physical platform.

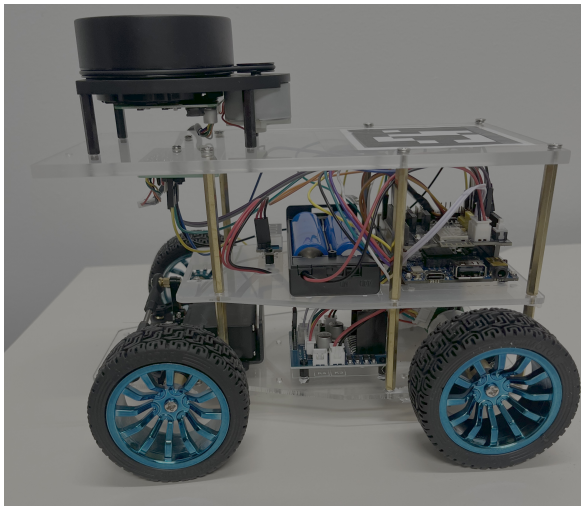

Fig. 1. The four-wheeled robot platform with the RPLidar A1 mounted on the top deck.

B. Euclidean Distance Transform Map

The room boundary is represented as a closed polygon of wall segments. This polygon is rasterized onto a uniform grid of cell size c=0.1 m. Interior cells are set to 1 (free) and wall/exterior cells to 0 (occupied).

The EDT is applied to this binary occupancy grid:

$$d(i,j) = \min_{(i',j')\in\mathcal{W}} \|(i,j)-(i',j')\|_2 \cdot c \qquad (1)$$

where $\mathcal{W}$ is the set of occupied cells. The scalar $d(i,j)$ encodes the straight-line clearance in metres to the nearest wall and is precomputed once via `scipy.ndimage.distance_transform_edt`. The surveyed room ($\approx$6.35 m $\times$ 5.25 m) yields a 64×53 grid.

Figure 2 shows the resulting clearance field and the derived MPPI wall cost (described in Section II-E). The clearance field peaks at approximately 1.7 m near the room centre and smoothly decays to zero at every wall boundary. The wall cost field shows the exponential barrier used by MPPI to repel trajectories from obstacles.

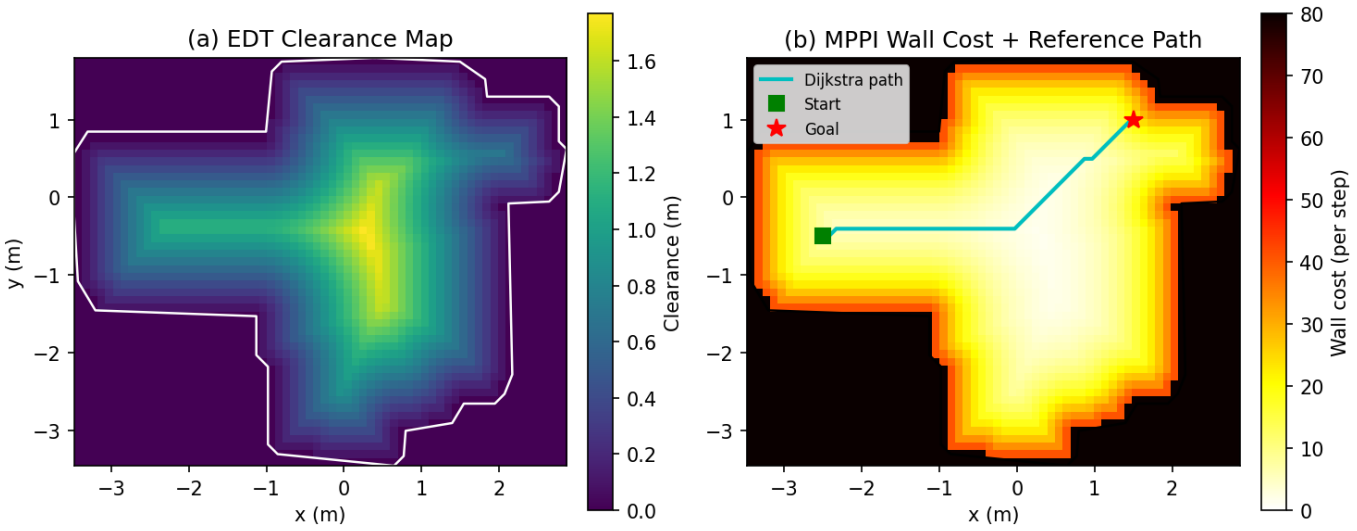

Fig. 2. (a) EDT clearance map: brighter colours indicate larger distance to the nearest wall. (b) MPPI wall cost field with an example Dijkstra reference path (cyan) from a start (green) to a goal (red). The cost field is saturated at 80 for display; cells inside the wall boundary carry a hard penalty of 1000.

C. Dijkstra Reference Path

A wall-clearance-penalized cost is assigned to each directed edge of the 8-connected grid graph:

$$w_{\text{edge}} = \frac{\ell_{\text{step}}}{1 - e^{-d_{\min}/\alpha}} \tag{2}$$

where ℓ_{step} is the Euclidean step length (c for cardinal, $c\sqrt{2}$ for diagonal), $d_{\min}$ is the minimum EDT clearance of the two endpoint cells, and α=5.0 m controls the decay sharpness. In open space ($d_{\min} \gg \alpha$) the denominator approaches 1 and the edge weight reduces to the Euclidean step; near walls ($d_{\min} \to 0$) the denominator collapses and the weight diverges, forcing paths to route away from obstacles.

The graph is pre-built as a sparse CSR matrix. When the operator sets a goal, a single call to `scipy.sparse.csgraph.dijkstra` from the robot's current PF mean position produces the globally optimal clearance-weighted reference path $\mathcal{P} = \{(x_k^p, y_k^p)\}_{k=1}^{P}$ in approximately 150–200 ms. Because path planning runs only once per goal assignment, this cost does not affect real-time operation.

D. Particle Filter Localization

A Monte Carlo Particle Filter [1] maintains N=1,000 weighted particles $\{(\mathbf{x}_t^{[i]}, w_t^{[i]})\}$ representing the posterior $p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$. Each particle is propagated via the robot's motion model using the encoder increment Δe and steering angle α_t; particles are then reweighted by a Gaussian likelihood over 50 LiDAR rays ray-cast against the map:

$$w_t^{[i]} \propto \prod_k \exp\left(-\frac{(r_k - r_k^{[i]})^2}{2\sigma_r^2}\right), \quad \sigma_r^2 = 0.1\,\text{m}^2 \tag{3}$$

Low-variance systematic resampling is applied every cycle. If the maximum weight falls below 10^{-9} for five consecutive cycles the particle set is reinitialized uniformly over free space. The PF mean $(\bar{x}, \bar{y}, \bar{\theta})$ is passed to MPPI as the current state.

E. MPPI Control

MPPI [2] is a sampling-based receding-horizon controller. At each iteration it samples K perturbed control sequences, rolls them forward through the motion model, scores them with a composite cost, and updates the nominal sequence via an importance-weighted average.

1) Trajectory Sampling: At each iteration K=100 steering sequences are sampled by perturbing the warm-started nominal:

$$\tilde{u}_{k,t}^s = \text{clip}(U_t^{s,\text{nom}} + \epsilon_{k,t},\ -20^\circ,\ 20^\circ), \quad \epsilon_{k,t} \sim \mathcal{N}(0,\ \sigma_{\text{steer}}^2) \tag{4}$$

with σ_{steer}=10°. The encoder component is held at the nominal rate $\bar{e}$=85.5 counts/step. All K trajectories $\boldsymbol{\tau}_k \in \mathbb{R}^{(T+1)\times 3}$ are integrated simultaneously using vectorized NumPy operations, with a single Python loop only over the T=60 time steps for the heading integration.

2) Cost Function: Each trajectory is scored by:

$$\mathcal{C}_k = \mathcal{C}_k^{\text{wall}} + \mathcal{C}_k^{\text{path}} + \mathcal{C}_k^{\text{head}} + \mathcal{C}_k^{\text{goal}} \tag{5}$$

Wall cost — penalizes obstacle proximity via the EDT (see Fig. 2b):

$$\mathcal{C}_k^{\text{wall}} = \sum_{t=1}^{T} \begin{cases} 1000 & d(x_{k,t}, y_{k,t}) \le 0 \\ \lambda_w\, e^{-d(x_{k,t},y_{k,t})/0.5} & \text{otherwise} \end{cases} \tag{6}$$

with λ_w=50. The hard barrier of 1000 inside wall cells strongly discourages any trajectory that enters occupied space.

Path cost — penalizes lateral deviation from $\mathcal{P}$:

$$\mathcal{C}_k^{\text{path}} = \lambda_p \sum_{t=1}^{T} \min_j \left\| (x_{k,t}, y_{k,t}) - (x_j^p, y_j^p) \right\|_2 \tag{7}$$

with λ_p=1.0.

Heading cost — penalizes misalignment with the path tangent (λ_h=1.5).

Goal cost — penalizes terminal distance to the goal:

$$\mathcal{C}_k^{\text{goal}} = \lambda_g \left\| (x_{k,T}, y_{k,T}) - \mathbf{x}^{\text{goal}} \right\|_2, \quad \lambda_g{=}100 \tag{8}$$

3) Control Update and Stopping: Trajectories are weighted by a softmax with temperature α=0.1 and the nominal sequence updated as their weighted average, with the steering component clamped to $[\pm 20°]$. The first element is transmitted; the sequence is shifted one step for warm-starting.

MPPI is deactivated when the PF mean reaches within r_{stop}=0.15 m of the goal, at which point a zero-velocity command is sent.

III. Experiment Design

A. Hardware Platform

The robot is a four-wheeled Ackermann-steered vehicle built on an RC car chassis. An Arduino GIGA R1 drives a brushed DC motor (speed) and steering servo (angle). An RPLidar A1 is mounted on the top deck. An ESP32 module provides WiFi, exchanging 10-byte UDP datagrams with the laptop at 100 ms intervals. The laptop runs a NiceGUI application with a 10 Hz control-loop timer and a decoupled 2 Hz map-visualization timer.

B. Map Acquisition

The room boundary was manually surveyed from LiDAR scans, producing 32 wall-segment endpoints forming a closed polygon of $\approx$6.35 m $\times$ 5.25 m.

C. Particle Filter Validation

Three initialization conditions were tested:

- **Good Init:** particles seeded near the true pose.
- **Bad Init:** particles seeded $\sim$1 m from truth.
- **Unknown Init:** particles uniformly distributed over all free cells.

Position spread σ_{xy} and heading spread σ_θ were logged at 10 Hz during a short driving circuit.

D. MPPI Navigation Trial

Once the PF converged (σ_{xy}<0.3 m), a goal coordinate was set via the GUI, triggering Dijkstra path planning and activating MPPI. The robot navigated autonomously until the stop condition triggered. Per-stage timing was logged throughout.

IV. Experimental Results

A. Particle Filter Convergence

Figure 3 shows particle spread versus time for all three initialization conditions. With a good initial pose the spread is near zero from the first scan. A bad initial pose causes a spike to $\approx$230 cm at $t\approx$0.5 s as the motion model propagates the erroneous estimate, before collapsing to below 5 cm within $\approx$1 s once LiDAR measurements correct the cloud. Unknown initialization begins at $\approx$83 cm and also converges within $\approx$1 s. Heading uncertainty σ_θ settles to 1–2° in all cases.

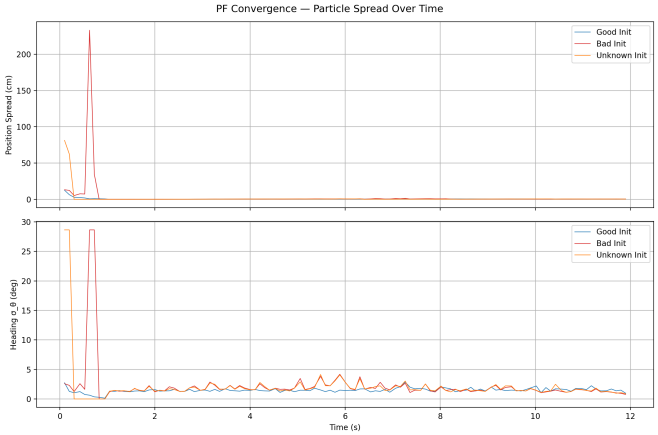

Fig. 3. Particle spread for three initialization conditions. Position spread (top) and heading spread (bottom) both converge within 1 s of robot motion.

Figure 4 shows the PF mean trajectories with 95% confidence ellipses. Ellipses collapse rapidly in the bad- and unknown-initialization cases, confirming fast convergence.

B. Control Loop Timing

Table I gives per-stage latencies during a full navigation run. The PF dominates at 38–40 ms due to ray-casting 50 rays for 1 000 particles. MPPI takes only 1.5–2.5 ms because all K=100 trajectories are propagated simultaneously with vectorized NumPy operations, avoiding the $K \times T$=6,000 individual Python calls of a naive loop. Total loop latency of 40–42 ms fits comfortably within the 100 ms UDP cycle.

TABLE I
Per-stage control loop timing (50 LiDAR rays, K=100, T=60)

Stage	Mean (ms)	Max (ms)
UDP Receive	$<$ 0.1	0.1
Particle Filter	38.7	73.6
MPPI	1.8	2.5
UDP Send	0.1	0.1
Total	**40.6**	**75.3**

C. MPPI Cost Tuning

The wall cost parameters had the largest impact on navigation safety. An initial configuration of λ_w=5 with a 0.3 m decay produced near-flat cost variation away from walls, causing the robot to clip corners. Increasing to λ_w=50 with a 0.5 m decay created a steep barrier visible in Fig. 2b, preventing sampled trajectories from approaching the wall boundary. The goal weight λ_g was raised to 100 to overcome competition between path and heading costs near the goal. Table II lists the final parameters.

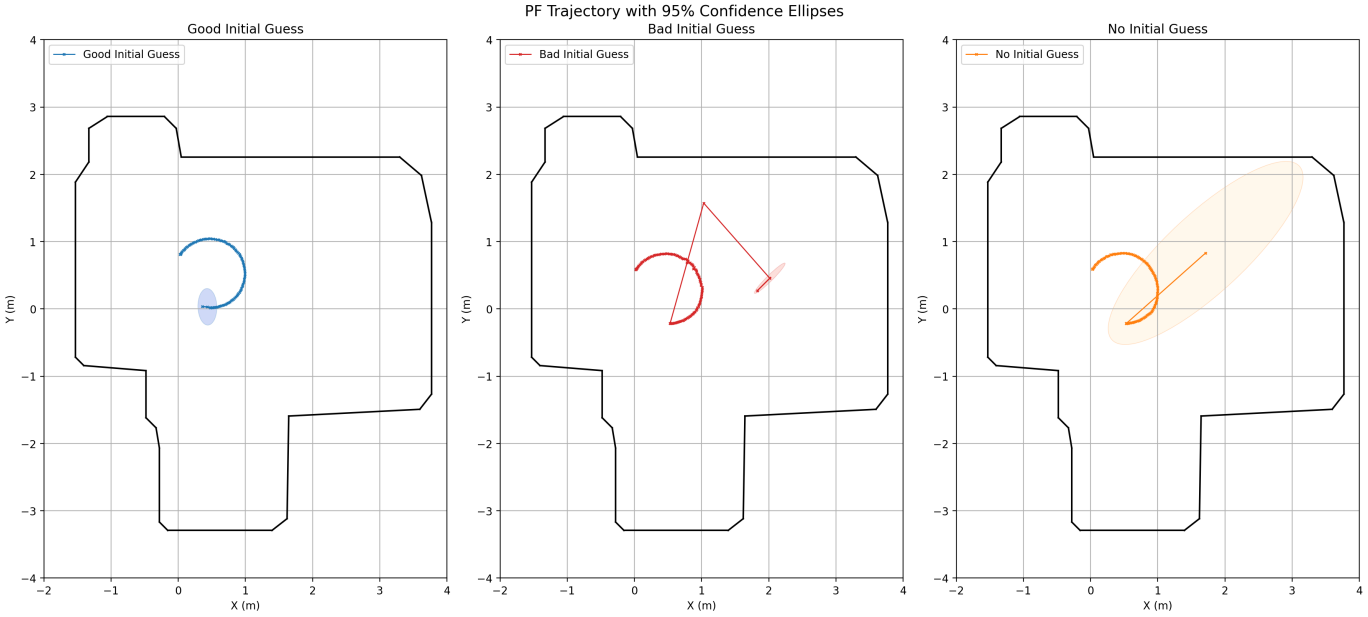

Fig. 4. PF mean trajectories with 95% confidence ellipses for good, bad, and unknown initial conditions. The black polygon is the room boundary.

TABLE II
FINAL MPPI HYPERPARAMETERS

Parameter	Symbol	Value
Trajectories	K	100
Horizon	T	60 steps
Step time	Δt	0.1 s
Temperature	α	0.1
Steering noise	σ_{steer}	10°
Wall weight	λ_w	50
Wall decay radius	—	0.5 m
Path weight	λ_p	1.0
Heading weight	λ_h	1.5
Goal weight	λ_g	100
Goal-stop radius	r_{stop}	0.15 m

V. CONCLUSION

A complete autonomous navigation pipeline was implemented and validated on a four-wheeled robot. The EDT map provides a compact, globally consistent representation of wall clearance that simultaneously drives Dijkstra path planning and the MPPI obstacle-avoidance cost. The Particle Filter converged from arbitrary initial positions within $\approx$1 s of motion with final heading uncertainty below 2°. The vectorized MPPI implementation completed each iteration in under 2.5 ms, leaving the 100 ms control budget dominated by PF ray-casting at 38–40 ms.

Key tuning insight: the wall cost decay radius is the most influential MPPI parameter. A radius below $\approx$0.5 m produces nearly flat cost across the navigable area, making the controller indifferent to wall proximity. Future work could replace the static Dijkstra reference with online replanning when the LiDAR detects transient obstacles, and reduce PF cost by porting the ray-caster to a compiled NumPy/Cython extension.

REFERENCES

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *Proc. IEEE ICRA*, Stockholm, Sweden, 2016, pp. 1433–1440.

[3] A. Meijster, J. B. T. M. Roerdink, and W. H. Hesselink, "A general algorithm for computing distance transforms in linear time," in *Mathematical Morphology and its Applications to Image and Signal Processing*, Springer, 2000, pp. 331–340.

[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[5] C. Clark, "Lab 05 — Autonomous Navigation," ROB-GY 6213 Robot Localization and Navigation, New York University, 2026.

[6] Imaginelenses, "NYU_ROB_GY_6213," GitHub, [Online]. https://github.com/imaginelenses/NYU_ROB_GY_6213